DATABASE MANAGEMENT SYSTEMS

Complete Study Guide

Covers: Architecture • Relational Model • ER Model • Normalization
Relational Algebra • SQL • Transactions • Concurrency Control

Unit 1: Introduction & Architecture

1.1 What is a DBMS?

A Database Management System (DBMS) is software that manages a collection of interrelated data and a set of programs to access that data. It serves as an interface between users and the database, providing efficient, reliable, and secure data management.

Real-Life Examples

- Banking: Account balances, transaction records, loan data
- E-Commerce (Amazon/Flipkart): Product catalog, orders, customer profiles
- Social Media (Facebook/Instagram): User posts, friend connections, likes
- Airlines: Flight schedules, seat reservations, passenger records
- Hospitals: Patient records, doctor schedules, billing
- Universities: Student enrollment, grades, course registrations

Data vs. Information vs. Database

Term	Definition	Example
Data	Raw, unprocessed facts	12345, John, 95
Information	Processed, meaningful data	Student John scored 95%
Database	Organized collection of related data	University DB with Students, Courses tables
DBMS	Software to manage the database	MySQL, Oracle, PostgreSQL

1.2 Advantages of DBMS over File Systems

Problem in File System	Solution in DBMS
Data Redundancy & Inconsistency	Centralized data eliminates duplication
Difficulty in Accessing Data	Query languages (SQL) for easy retrieval
Data Isolation	Integrated data accessible to all authorized users
Integrity Problems	Integrity constraints enforced by DBMS
Atomicity Problems	Transactions ensure all-or-nothing operations
Concurrent Access Anomalies	Concurrency control mechanisms
Security Problems	Role-based access control (RBAC)
No Crash Recovery	DBMS provides backup and recovery mechanisms

1.3 DBMS Architecture

2-Tier Architecture

In 2-tier architecture, the client directly communicates with the database server. The application logic resides on the client side.

```
┌──────────────────────┐        ┌──────────────────────┐
│  CLIENT TIER  │ ◀──────▶ │  DATABASE TIER  │
│ (Application + │  Direct  │  (DBMS + Data) │
│   UI Logic)  │ Connect  │              │
└──────────────────────┘        └──────────────────────┘
```

Example: Desktop apps connecting directly to MySQL

- Pros: Simple, fast for small applications
- Cons: Poor scalability, business logic on client (security risk), tight coupling

3-Tier Architecture

Introduces a middle application/business logic layer between client and database. Most modern web applications follow this.

```
┌──────────────┐   ┌──────────────────┐   ┌──────────────────┐
│  CLIENT  │◀──▶│  APPLICATION  │◀──▶│   DATABASE   │
│  TIER    │   │ SERVER TIER  │   │   TIER     │
│(Browser/ │   │(Business Logic│   │(DBMS + Data) │
│ Mobile)  │   │ + API Layer) │   │            │
└──────────────┘   └──────────────────┘   └──────────────────┘
```

Example: React frontend → Node.js server → PostgreSQL

- Pros: Scalable, secure (DB hidden from client), easy maintenance, load balancing
- Cons: More complex, slightly higher latency

1.4 Three-Level Schema Architecture (ANSI/SPARC)

Also called the 3-schema architecture. It separates how data is stored from how it appears to users — enabling data independence.

Level	Name	Description	Who Uses It
Level 3 (Top)	External / View Level	How different users see the data. Each user gets a custom view.	End Users, Application Programs
Level 2 (Middle)	Conceptual / Logical Level	Describes the whole database structure — tables, relationships, constraints.	DBA, Developers

Level 1 (Bottom)	Internal / Physical Level	How data is physically stored — file formats, indexes, storage paths.	DBMS Engine, Storage System

> Example: A University Database
>
> External View (Student): Can only see their own grades and courses.
>
> External View (Admin): Can see all student records and transcripts.
>
> Conceptual Level: Tables: Student(ID, Name, GPA), Course(CID, Name, Credits)
>
> Internal Level: Data stored as B-tree indexed files on SSD.

1.5 Data Independence

Logical Data Independence

The ability to change the conceptual (logical) schema without changing the external schema (user views) or application programs.

- Example: Adding a new column 'Phone' to the Student table should not break existing applications that only query Name and GPA.
- Achieved through: External-Conceptual mapping

Physical Data Independence

The ability to change the internal (physical) schema without changing the conceptual schema.

- Example: Moving the database from HDD to SSD, or changing index structure from B-tree to Hash, should not affect logical design.
- Achieved through: Conceptual-Internal mapping

📌 **Note:** Physical independence is easier to achieve than logical independence. Most commercial DBMS provide good physical independence.

Unit 2: Relational Model & Keys

2.1 The Relational Model

Proposed by E.F. Codd in 1970. Data is represented as a collection of relations (tables). Each relation has rows (tuples) and columns (attributes).

Formal Term	Common Term	Meaning
Relation	Table	A 2D structure of rows and columns
Tuple	Row / Record	A single data entry
Attribute	Column / Field	A property of the entity
Domain	Data Type	Allowed values for an attribute
Degree	Number of columns	How many attributes in a relation
Cardinality	Number of rows	How many tuples in a relation

2.2 Integrity Constraints

Domain Constraint

Every attribute value must be from the domain of that attribute. E.g., Age must be an integer, not a string.

Entity Integrity Constraint

The primary key of a table cannot be NULL. Every row must be uniquely identified.

Referential Integrity Constraint

A foreign key in one table must match a primary key value in the referenced table, or be NULL. Ensures no orphan records.

Key Constraint

No two tuples in a relation can have the same combination of key attribute values.

2.3 Database Keys — Deep Dive

Super Key

Any set of one or more attributes that can uniquely identify a tuple in a relation. Super keys may contain redundant attributes.

> Table: Student(RollNo, Name, Email, Phone)
>
> Super Keys: {RollNo}, {Email}, {Phone}, {RollNo, Name}, {RollNo, Email},

{RollNo, Name, Email}, {RollNo, Name, Email, Phone} ... (many more)

Every superset of a super key is also a super key.

Candidate Key

A minimal super key — a super key with no redundant attributes. Every attribute in a candidate key is necessary to ensure uniqueness.

From the Student table above:

Candidate Keys: {RollNo}, {Email}, {Phone}

{RollNo, Name} is NOT a candidate key because RollNo alone is sufficient.

A table can have multiple candidate keys.

- Every candidate key is a super key, but not every super key is a candidate key.

Primary Key (PK)

One candidate key chosen by the designer to uniquely identify tuples. Properties: Unique, Not NULL, Stable (rarely changes).

- A table can have only ONE primary key.
- Can be a single attribute (simple PK) or multiple attributes (composite PK).
- Example: RollNo is typically chosen as PK for Student table.

Alternate Key

Candidate keys that are NOT selected as the primary key. Still unique, but not the primary identifier.

- Example: If RollNo is PK, then Email and Phone are alternate keys.

Foreign Key (FK)

An attribute (or set of attributes) in one table that refers to the Primary Key of another table. It establishes a link between two tables.

Table: Enrollment(StudentID, CourseID, Grade)

Student(RollNo, Name, Email)

Course(CourseID, CourseName, Credits)

StudentID in Enrollment → refers to RollNo (PK) in Student

CourseID in Enrollment → refers to CourseID (PK) in Course

StudentID and CourseID are FOREIGN KEYS in Enrollment.

2.4 Foreign Key Deep Dive & Referential Integrity

Insert Operation

- Inserting in parent table: Always allowed (no FK concern).
- Inserting in child table: The FK value must exist in the parent table's PK. Otherwise, referential integrity is violated.

> INSERT INTO Enrollment VALUES (999, 'CS101', 'A');
>
> → FAILS if StudentID=999 does not exist in Student table.

Delete Operation

- Deleting from child table: Always allowed.
- Deleting from parent table: If referenced rows exist in child table, it VIOLATES referential integrity.

Resolution options:

- ON DELETE RESTRICT: Prevent deletion if child rows exist (default).
- ON DELETE CASCADE: Automatically delete all child rows.
- ON DELETE SET NULL: Set the FK in child rows to NULL.
- ON DELETE SET DEFAULT: Set FK to a default value.

Update Operation

- Updating PK in parent: Same problem as delete — child rows may become orphans.
- ON UPDATE CASCADE: Updates FK in child table to match new PK value.

ON DELETE CASCADE — Example

```
CREATE TABLE Student (
    RollNo INT PRIMARY KEY,
    Name VARCHAR(50)
);

CREATE TABLE Enrollment (
    StudentID INT,
    CourseID VARCHAR(10),
    FOREIGN KEY (StudentID) REFERENCES Student(RollNo)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);

-- When you delete Student with RollNo=101,
-- ALL rows in Enrollment with StudentID=101 are AUTOMATICALLY deleted.
```

Unit 3: Entity-Relationship (ER) Model

3.1 Introduction to ER Model

The ER (Entity-Relationship) Model is a high-level conceptual data model proposed by Peter Chen in 1976. It is used to visualize the database design before implementation.

- Entity: A real-world object distinguishable from other objects. E.g., Student, Course, Employee.
- Entity Set: A collection of similar entities. E.g., all students in a university.
- Relationship: An association among two or more entities. E.g., Student ENROLLS IN Course.
- Relationship Set: A collection of similar relationships.

3.2 Types of Attributes

Attribute Type	Description	Example
Simple (Atomic)	Cannot be divided further	Age, RollNo
Composite	Can be divided into sub-parts	Name → (FirstName, LastName); Address → (Street, City, PIN)
Single-Valued	Holds only one value per entity	DateOfBirth
Multi-Valued	Holds multiple values (shown with double ellipse)	PhoneNumbers: {9876543210, 9123456789}
Derived	Value computed from other attributes (shown with dashed ellipse)	Age derived from DateOfBirth
Stored	Attribute from which derived attributes are computed	DateOfBirth
NULL	Attribute value is unknown or inapplicable	MiddleName may be NULL
Key Attribute	Uniquely identifies an entity (underlined in ER diagram)	RollNo for Student
Partial Key	Partially identifies a weak entity (dashed underline)	OrderItemNo in OrderItem

3.3 Types of Relationships

One-to-One (1:1)

Each entity in A is associated with at most one entity in B, and vice versa.

> Example: Each Employee manages at most one Department.
>
> Each Department is managed by at most one Employee.
>
> Employee ———————— manages ———————— Department

```
1                    1
```

One-to-Many (1:M)

One entity in A can be associated with many entities in B, but each entity in B is associated with at most one entity in A.

```
Example: One Customer can place many Orders.
        Each Order belongs to exactly one Customer.

Customer ———————— places ———————— Order
  1                    M
```

Many-to-Many (M:N)

Entities in A can be associated with many entities in B, and vice versa. Requires a junction/associative table in implementation.

```
Example: A Student can enroll in many Courses.
        A Course can have many Students enrolled.

Student ———————— enrolls in ———————— Course
  M                    N

Implementation: Enrollment(StudentID, CourseID, Grade) — junction table
```

3.4 Participation Constraints

Total Participation (Mandatory)

Every entity in the set must participate in at least one relationship. Shown by a double line in ER diagram.

- Example: Every Student MUST enroll in at least one Course (if enrollment is mandatory).

Partial Participation (Optional)

Some entities may not participate in any relationship. Shown by a single line.

- Example: Not every Employee manages a Department (some are just employees).

3.5 Weak Entity Sets

A weak entity is an entity that does not have sufficient attributes to form a primary key on its own. It depends on a strong (owner) entity for its existence and identification.

- Shown with a double rectangle in ER diagrams.

- Its relationship with the strong entity is called an Identifying Relationship (shown with a double diamond).
- Has a Partial Key (dashed underline) — unique only within the context of the strong entity.

> Example: OrderItem is a weak entity depending on Order.
>
> Order (Strong Entity) ═══════ contains ═══════ OrderItem (Weak Entity)
>
> OrderID (PK) ItemNo (Partial Key)
>
> Quantity
>
> Price
>
> Full PK of OrderItem = (OrderID, ItemNo)
>
> If Order is deleted, all its OrderItems are also deleted (ON DELETE CASCADE).

3.6 Minimizing Tables in ER Model

When converting an ER diagram to tables, we can minimize the number of tables:

- 1:1 relationship: Can merge into one table (add FK in either table, or merge both).
- 1:M relationship: Add FK on the 'many' side. No separate relationship table needed.
- M:N relationship: Always requires a separate junction/relationship table.
- Weak entity: Merge with its identifying relationship — the weak entity table includes the PK of the strong entity.
- Multi-valued attribute: Requires a separate table (attribute table referencing the entity's PK).

Unit 4: Normalization & Functional Dependencies

4.1 Functional Dependencies (FD)

A functional dependency X → Y means: if two tuples agree on the values of attributes X, they must also agree on values of Y. We say 'X functionally determines Y' or 'Y is functionally dependent on X'.

```
Notation: X → Y  (X determines Y)


In Student(RollNo, Name, Dept, DeptHead):
  RollNo → Name      (Each roll number has one name)
  RollNo → Dept      (Each student belongs to one department)
  Dept → DeptHead    (Each department has one head)
  RollNo → DeptHead  (Derived via transitivity)
```

Types of Functional Dependencies

- Trivial FD: X → Y where Y ⊆ X. E.g., {A, B} → A. Always holds.
- Non-trivial FD: Y ⊄ X. Actually provides new information.
- Completely Non-trivial: X ∩ Y = ∅.

Armstrong's Axioms (Inference Rules)

Rule	Name	Statement
A1	Reflexivity	If Y ⊆ X, then X → Y (trivial FD)
A2	Augmentation	If X → Y, then XZ → YZ for any Z
A3	Transitivity	If X → Y and Y → Z, then X → Z
Derived: Union	Union	If X → Y and X → Z, then X → YZ
Derived: Decomposition	Decomposition	If X → YZ, then X → Y and X → Z
Derived: Pseudo-transitivity	Pseudo-transitivity	If X → Y and WY → Z, then WX → Z

Closure of an Attribute Set (X+)

The closure of X (X+) is the set of all attributes that can be functionally determined by X using the given FDs.

```
Algorithm to find X+:
1. Start with X+ = X
2. Repeat until no change:
```

For each FD A → B in F:

If A ⊆ X+, then add B to X+

3. Return X+

Example: R(A,B,C,D,E), FDs: A→B, B→C, C→D, D→E

Find A+:

Start: {A}

A→B: {A,B}

B→C: {A,B,C}

C→D: {A,B,C,D}

D→E: {A,B,C,D,E}

A+ = {A,B,C,D,E} → A is a candidate key!

Minimal Cover (Canonical Cover)

A minimal cover Fc of a set of FDs F is an equivalent set of FDs that has no redundancy. Steps to find it:

- Step 1: Decompose all RHS to single attributes. E.g., A → BC becomes A → B and A → C.
- Step 2: Remove extraneous attributes from LHS. For each FD XA → Y, check if X → Y holds without A. If yes, replace XA → Y with X → Y.
- Step 3: Remove redundant FDs. For each FD X → Y, check if X → Y can be derived from the remaining FDs. If yes, remove it.

Dependency Preserving Decomposition

A decomposition of R into R1, R2, ... Rk is dependency preserving if for every FD X → Y in F, there exists some Ri such that both X and Y appear in Ri. In other words, all original FDs can be checked locally without needing a join.

Lossless vs. Lossy Decomposition

A decomposition is lossless (non-additive) if the original relation can be perfectly reconstructed by joining the decomposed relations. No spurious (extra false) tuples are generated.

Test for Lossless Decomposition (2 relations):

R is decomposed into R1 and R2.

The decomposition is lossless if:

(R1 ∩ R2) → R1 OR (R1 ∩ R2) → R2

i.e., the common attributes form a super key in at least one of R1 or R2.

Example: R(A,B,C), FD: A→B

Decompose into R1(A,B) and R2(A,C).

R1 ∩ R2 = {A}. Does A → AB? A → A (trivial) + A → B (given) = Yes!

Lossless ✓

4.2 Anomalies — Why Normalization is Needed

Anomalies arise from storing related data in one table without proper normalization:

Anomaly Type	Description	Example
Insertion Anomaly	Cannot insert data without other unrelated data	Cannot add a new Course until a Student enrolls (if stored in one table)
Deletion Anomaly	Deleting a record unintentionally loses other data	Deleting the last student in a department also deletes the department info
Update Anomaly	Updating one value requires updating multiple rows	If a Dept Head changes, need to update every student's row in that department

4.3 Normal Forms

First Normal Form (1NF)

A relation is in 1NF if:

- All attributes contain only atomic (indivisible) values.
- No multi-valued or composite attributes.
- Each row is uniquely identifiable (has a primary key).

```
VIOLATES 1NF (multi-valued attribute):          SATISFIES 1NF:

┌──────────┬──────────────┐                    ┌──────────┬─────────┐
│ Student  │ Courses      │                    │ Student  │ Course  │
├──────────┼──────────────┤                    ├──────────┼─────────┤
│   101    │ Math,Physics │  ──────────────►   │   101    │ Math    │
│   102    │ CS,Math      │                    │   101    │ Physics │
└──────────┴──────────────┘                    │   102    │ CS      │
                                               │   102    │ Math    │
                                               └──────────┴─────────┘
```

Second Normal Form (2NF)

A relation is in 2NF if it is in 1NF and every non-key attribute is fully functionally dependent on the entire primary key (no partial dependency).

- Partial dependency: A non-prime attribute depends on PART of a composite primary key.
- 2NF only applies when there is a composite primary key.

```
Table: Enrollment(StudentID, CourseID, StudentName, CourseName, Grade)
PK: (StudentID, CourseID)

Partial Dependencies:
  StudentID → StudentName   (depends only on StudentID, not full PK)
  CourseID → CourseName     (depends only on CourseID, not full PK)

Fix — Decompose into:
  Student(StudentID, StudentName)
  Course(CourseID, CourseName)
  Enrollment(StudentID, CourseID, Grade)  ← Only Grade fully depends on PK
```

Third Normal Form (3NF)

A relation is in 3NF if it is in 2NF and there is no transitive dependency — no non-prime attribute depends on another non-prime attribute.

```
Table: Student(RollNo, Dept, DeptHead)
PK: RollNo

Transitive Dependency: RollNo → Dept → DeptHead
DeptHead depends on Dept (non-prime), not directly on RollNo.

Fix — Decompose into:
  Student(RollNo, Dept)
  Department(Dept, DeptHead)
```

Formal definition of 3NF: For every FD X → A in R, at least one must hold:

- X is a super key of R, OR
- A is a prime attribute (part of some candidate key).

Boyce-Codd Normal Form (BCNF)

BCNF is a stricter version of 3NF. For every FD X → Y, X must be a super key. BCNF removes all anomalies due to FDs.

- BCNF implies 3NF, but 3NF does NOT always imply BCNF.

- BCNF decomposition may not always be dependency preserving.

> Table: Teaches(Student, Course, Teacher)
>
> Assumption: Each teacher teaches one course; each student per course has one teacher.
>
> FDs: (Student, Course) → Teacher Teacher → Course
>
> Candidate Keys: {Student, Course}, {Student, Teacher}
>
> For FD Teacher → Course: Teacher is NOT a super key. Violates BCNF!
>
> Fix — Decompose into:
>
> TeacherCourse(Teacher, Course) ← Teacher is PK
>
> StudentTeacher(Student, Teacher) ← (Student, Teacher) is PK

Fifth Normal Form (5NF) / Project-Join Normal Form (PJNF)

A relation is in 5NF if it cannot be further decomposed without information loss. It addresses JOIN dependencies — cases where a table can be decomposed into three or more tables but not two without loss.

- Rarely needed in practice but important for theoretical completeness.
- Example: A supplier-part-project table where a supplier supplies a part to a project, but any pair combination could be split and rejoined.

Normal Form	Rule / Condition
1NF	Atomic values; no repeating groups
2NF	1NF + No partial dependencies (non-key attrs depend on entire PK)
3NF	2NF + No transitive dependencies (non-key attrs only depend on PK)
BCNF	3NF + For every FD X→Y, X must be a super key
4NF	BCNF + No multi-valued dependencies
5NF	4NF + No join dependencies

Unit 5: Relational Algebra & Calculus

5.1 Relational Algebra

Relational Algebra is a procedural query language that takes relations as input and returns a relation as output. It forms the theoretical foundation of SQL.

Fundamental Operations

1. Selection (σ) — Filter Rows

σ_condition(R) — Returns tuples from R that satisfy the condition.

> σ_Age > 20(Student) → Returns all students older than 20
>
> σ_Dept = 'CS' AND CGPA > 8.0(Student) → CS students with high GPA

2. Projection (π) — Select Columns

π_attr1,attr2,...(R) — Returns only specified columns (with duplicate elimination).

> π_Name, Dept(Student) → Returns only Name and Dept columns

3. Cross Product / Cartesian Product (×)

R × S — Combines every tuple of R with every tuple of S. If R has m rows and S has n rows, result has m×n rows.

> Student × Course → All combinations of students and courses
>
> (Usually combined with selection to get meaningful results — this is a Join)

4. Union (∪)

R ∪ S — Returns all tuples in R or S or both. R and S must be union-compatible (same number of attributes with compatible domains).

5. Set Difference (−)

R − S — Returns tuples in R that are NOT in S. R and S must be union-compatible.

> AllStudents − EnrolledStudents → Students who are NOT enrolled in any course

6. Rename (ρ)

ρ_new_name(old_attrs → new_attrs)(R) — Renames the relation and/or its attributes.

Derived Operations

7. Intersection (∩)

R ∩ S = R − (R − S) — Tuples that appear in both R and S.

8. Natural Join (⋈)

Joins R and S on common attributes (matching column names), performs equi-join, and removes duplicate columns. Most commonly used join.

> Student ⋈ Enrollment → Combines on matching StudentID, auto-detects common attrs

9. Division (÷)

R ÷ S — Returns tuples from R that are associated with ALL tuples in S. Used for 'for all' queries.

> Query: Find students enrolled in ALL CS courses.
>
> Enrollment ÷ CSCourses → gives StudentIDs enrolled in every CS course

Operator	Symbol	SQL Equivalent	Description
Selection	σ	WHERE clause	Filter rows based on condition
Projection	π	SELECT columns	Select specific columns
Cross Product	×	FROM A, B	Cartesian product of two tables
Union	∪	UNION	All rows from both (compatible)
Set Difference	−	EXCEPT / MINUS	Rows in R not in S
Rename	ρ	AS alias	Rename relation or attributes
Natural Join	⋈	NATURAL JOIN	Join on common attributes
Division	÷	(complex subquery)	Find tuples related to all tuples in divisor

5.2 Tuple Relational Calculus (TRC)

TRC is a non-procedural query language. It specifies WHAT to retrieve, not HOW. Queries are in the form: { t | P(t) } where t is a tuple variable and P(t) is the predicate.

> Find all students with CGPA > 8.5:
>
> { t | t ∈ Student ∧ t.CGPA > 8.5 }
>
> Find student names enrolled in course 'CS101':
>
> { t.Name | t ∈ Student ∧ ∃e ∈ Enrollment (e.StudentID = t.RollNo ∧ e.CourseID = 'CS101') }

- ∃ (Existential quantifier): There exists at least one tuple satisfying the condition.
- ∀ (Universal quantifier): All tuples must satisfy the condition.

Unit 6: Database Joins

6.1 Why Joins?

In a normalized database, data is spread across multiple tables. Joins let us combine rows from two or more tables based on related columns, reconstructing the meaningful combined data.

> Example Tables Used Throughout This Section:
>
> Employee: EmpID | Name | DeptID
>
> ------+--------+-------
>
> 101 | Alice | 10
>
> 102 | Bob | 20
>
> 103 | Carol | 30
>
> 104 | Dave | NULL
>
> Department: DeptID | DeptName
>
> -------+----------
>
> 10 | HR
>
> 20 | IT
>
> 30 | Finance
>
> 40 | Marketing

6.2 Types of Joins

INNER JOIN (Equi Join / Natural Join)

Returns only rows where the join condition is TRUE in both tables. Non-matching rows are excluded.

```
SELECT E.Name, D.DeptName
FROM Employee E INNER JOIN Department D ON E.DeptID = D.DeptID;

-- Result: Alice-HR, Bob-IT, Carol-Finance  (Dave excluded — NULL DeptID)
```

- Equi Join: Uses = operator in join condition.
- Natural Join: Automatically joins on ALL columns with the same name, removes duplicate columns.

LEFT OUTER JOIN

Returns ALL rows from the LEFT table, and matched rows from the RIGHT table. Unmatched right side = NULL.

```
SELECT E.Name, D.DeptName
FROM Employee E LEFT JOIN Department D ON E.DeptID = D.DeptID;

-- Result: Alice-HR, Bob-IT, Carol-Finance, Dave-NULL
-- Dave is included even though DeptID is NULL
```

RIGHT OUTER JOIN

Returns ALL rows from the RIGHT table, and matched rows from the LEFT table. Unmatched left side = NULL.

```
SELECT E.Name, D.DeptName
FROM Employee E RIGHT JOIN Department D ON E.DeptID = D.DeptID;

-- Result: Alice-HR, Bob-IT, Carol-Finance, NULL-Marketing
-- Marketing included even though no employee belongs to it
```

FULL OUTER JOIN

Returns ALL rows from BOTH tables. Unmatched rows on either side get NULL for the other table's columns.

```
SELECT E.Name, D.DeptName
FROM Employee E FULL OUTER JOIN Department D ON E.DeptID = D.DeptID;

-- Result: Alice-HR, Bob-IT, Carol-Finance, Dave-NULL, NULL-Marketing
```

SELF JOIN

A table joins with itself. Useful for hierarchical or comparative queries within the same table.

```
-- Find employees and their managers (both in Employee table)
SELECT E.Name AS Employee, M.Name AS Manager
FROM Employee E JOIN Employee M ON E.ManagerID = M.EmpID;
```

Join Type	Rows Returned	Unmatched Rows
INNER JOIN	Only matching rows from both tables	Excluded
LEFT JOIN	All rows from left + matched from right	Right side = NULL
RIGHT JOIN	All rows from right + matched from left	Left side = NULL
FULL OUTER JOIN	All rows from both tables	NULL on unmatched side
CROSS JOIN	Every row of A × every row of B	No condition — Cartesian product
SELF JOIN	Table joined with itself	Depends on join type used
NATURAL JOIN	Rows matching on all common column names	Excluded

Unit 7: Structured Query Language (SQL)

7.1 SQL Command Categories

Category	Full Name	Commands	Purpose
DDL	Data Definition Language	CREATE, ALTER, DROP, TRUNCATE, RENAME	Define/modify database structure
DML	Data Manipulation Language	SELECT, INSERT, UPDATE, DELETE	Manipulate data in tables
DCL	Data Control Language	GRANT, REVOKE	Control access permissions
TCL	Transaction Control Language	COMMIT, ROLLBACK, SAVEPOINT	Manage transactions

7.2 Table Operations

CREATE TABLE

```
CREATE TABLE Student (
    RollNo     INT PRIMARY KEY,
    Name       VARCHAR(100) NOT NULL,
    Email      VARCHAR(100) UNIQUE,
    Age        INT CHECK (Age >= 18 AND Age <= 60),
    DeptID     INT DEFAULT 1,
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID) ON DELETE SET NULL
);
```

ALTER TABLE

```
ALTER TABLE Student ADD COLUMN Phone VARCHAR(15);      -- Add column
ALTER TABLE Student DROP COLUMN Phone;                 -- Remove column
ALTER TABLE Student MODIFY COLUMN Name VARCHAR(200);   -- Change data type
ALTER TABLE Student RENAME COLUMN Age TO StudentAge;   -- Rename column
ALTER TABLE Student ADD CONSTRAINT uq_email UNIQUE(Email); -- Add constraint
```

DROP vs TRUNCATE vs DELETE

Command	What it removes	Can Rollback?	Resets Auto-Increment?	Condition (WHERE)?
DROP TABLE	Table structure + all data	No (DDL)	N/A — table gone	No
TRUNCATE TABLE	All rows (keeps structure)	No (DDL in most DBs)	Yes	No
DELETE FROM	Specific or all rows	Yes (DML)	No	Yes (optional)

7.3 Constraints

- NOT NULL: Column cannot have NULL values.
- UNIQUE: All values in column must be distinct. Multiple per table allowed.
- PRIMARY KEY: Unique + NOT NULL. Only one per table (can be composite).
- FOREIGN KEY: Enforces referential integrity to another table.
- CHECK: Enforces a condition on column values. E.g., CHECK(salary > 0).
- DEFAULT: Provides a default value if none is specified during insert.

7.4 Aggregate Functions & NULL Handling

Function	Purpose	NULL Handling
COUNT(*)	Counts all rows including NULLs	Includes NULL rows
COUNT(col)	Counts non-NULL values in column	Ignores NULLs
SUM(col)	Sum of all values	Ignores NULLs
AVG(col)	Average (SUM/COUNT of non-NULLs)	Ignores NULLs in numerator AND denominator
MIN(col)	Minimum value	Ignores NULLs
MAX(col)	Maximum value	Ignores NULLs

📌 **Note:** AVG ignores NULLs — AVG(col) = SUM(col) / COUNT(col), not total rows. This can be misleading!

7.5 GROUP BY, HAVING, and CTEs

GROUP BY

Groups rows with the same values in specified columns. Used with aggregate functions.

```
SELECT DeptID, COUNT(*) AS EmpCount, AVG(Salary) AS AvgSal
FROM Employee
GROUP BY DeptID;
```

HAVING

Filters groups after GROUP BY (like WHERE but for groups). Can use aggregate functions.

```
SELECT DeptID, AVG(Salary)
FROM Employee
GROUP BY DeptID
HAVING AVG(Salary) > 50000;
```

📌 **Note:** WHERE filters rows BEFORE grouping. HAVING filters groups AFTER aggregation.

WITH Clause (Common Table Expression — CTE)

CTEs create a named temporary result set usable within a query. Improves readability of complex queries.

```
WITH HighEarners AS (
    SELECT EmpID, Name, Salary
    FROM Employee
    WHERE Salary > 80000
)
SELECT * FROM HighEarners WHERE DeptID = 10;
```

7.6 SQL Operators

IN / NOT IN

```
SELECT * FROM Student WHERE DeptID IN (10, 20, 30);
SELECT * FROM Student WHERE DeptID NOT IN (SELECT DeptID FROM ClosedDepts);
```

EXISTS / NOT EXISTS

Returns TRUE if the subquery returns at least one row. More efficient than IN for correlated subqueries.

```
SELECT Name FROM Student S
WHERE EXISTS (
    SELECT 1 FROM Enrollment E WHERE E.StudentID = S.RollNo
);  -- Students who are enrolled in at least one course
```

LIKE Operator

```
SELECT * FROM Employee WHERE Name LIKE 'A%';   -- Names starting with A
SELECT * FROM Employee WHERE Name LIKE '%son'; -- Names ending with 'son'
SELECT * FROM Employee WHERE Name LIKE '_a%';  -- 2nd char is 'a'
-- % matches any sequence of characters
-- _ matches exactly one character
```

7.7 Subqueries

Non-Correlated Subquery

The inner query executes once and its result is used by the outer query. The inner query is independent.

```
-- Find employees earning more than average salary
SELECT Name FROM Employee
WHERE Salary > (SELECT AVG(Salary) FROM Employee);
```

Correlated Subquery

The inner query references the outer query and executes once FOR EACH ROW of the outer query. Slower but necessary for row-by-row comparisons.

```
-- Find employees who earn the max salary in their own department
SELECT E1.Name, E1.DeptID, E1.Salary
FROM Employee E1
WHERE E1.Salary = (
    SELECT MAX(E2.Salary)
    FROM Employee E2
    WHERE E2.DeptID = E1.DeptID  -- references outer query
```

```
);
```

Finding Nth Highest Salary

```
-- Method 1: Using LIMIT/OFFSET
SELECT DISTINCT Salary FROM Employee
ORDER BY Salary DESC LIMIT 1 OFFSET N-1;  -- Replace N with the rank

-- Method 2: Using subquery (for N=3rd highest)
SELECT MIN(Salary) FROM Employee
WHERE Salary IN (
    SELECT DISTINCT TOP 3 Salary FROM Employee ORDER BY Salary DESC
);

-- Method 3: Using DENSE_RANK()
SELECT Salary FROM (
    SELECT Salary, DENSE_RANK() OVER (ORDER BY Salary DESC) AS rnk
    FROM Employee
) ranked WHERE rnk = N;
```

7.8 SQL Query Execution Order

SQL queries are written in one order but executed in a different order:

Execution Step	Clause	What Happens
1	FROM / JOIN	Identify base tables; perform joins
2	WHERE	Filter individual rows
3	GROUP BY	Group the filtered rows
4	HAVING	Filter groups
5	SELECT	Choose columns; evaluate expressions
6	DISTINCT	Remove duplicate rows
7	ORDER BY	Sort the results
8	LIMIT / OFFSET	Restrict the number of output rows

📌 **Note:** This is why you CANNOT use a SELECT alias in a WHERE clause — WHERE executes before SELECT!

Unit 8: Transactions & Concurrency Control

8.1 What is a Transaction?

A transaction is a unit of program execution that accesses and possibly modifies various data items. It represents a logical unit of work (e.g., transferring money between two bank accounts).

```
Example: Transfer Rs 500 from Account A to Account B

T1: Read(A)        T3: Read(B)

T2: A = A - 500    T4: B = B + 500

   Write(A)          Write(B)


All four steps must succeed together. If T2 fails after T1, A is debited but B is never
credited — inconsistency!
```

8.2 ACID Properties

Property	Meaning	Ensured By
Atomicity	All operations in a transaction execute completely, or none execute at all. 'All or Nothing'.	Recovery Manager (undo log)
Consistency	A transaction brings the database from one valid state to another. All constraints must hold.	Application Logic + Integrity Constraints
Isolation	Concurrent transactions execute as if they were serial. Intermediate states are hidden.	Concurrency Control (Locking, MVCC)
Durability	Once a transaction commits, its changes are permanent even in case of system failure.	Recovery Manager (redo log / WAL)

8.3 Transaction States

```
      BEGIN
        │
        ▼
    [ACTIVE] ─────── execution ──────► [PARTIALLY COMMITTED]
        │                       │
        │ abort/fail              │ success
        │                       ▼
        └──────────────────────────────► [COMMITTED]
```

```
                              |
[FAILED] ◀—— error ————————————————┘
   |
   ▼
[ABORTED]  (Rollback + restart or kill)
```

8.4 Schedules

Serial vs. Parallel Schedules

- Serial Schedule: Transactions execute one after another with no interleaving. Always consistent but poor performance.
- Concurrent (Parallel) Schedule: Operations of multiple transactions are interleaved. Better performance but may cause inconsistencies.

Recoverable vs. Irrecoverable Schedules

A schedule is recoverable if: whenever transaction Ti reads a value written by Tj, then Tj must commit before Ti commits.

- Irrecoverable Schedule: Ti commits before Tj commits — if Tj later aborts, Ti read incorrect (dirty) data and already committed. Cannot undo!
- Recoverable Schedule: Ti commits only after Tj commits. We can rollback Ti if needed.

Cascading vs. Cascadeless Schedules

- Cascading Rollback: Aborting one transaction forces rollback of other transactions that read its uncommitted data. Expensive!
- Cascadeless Schedule: Transactions only read committed data. No cascading rollbacks needed.

📌 **Note:** Every cascadeless schedule is recoverable, but not every recoverable schedule is cascadeless.

8.5 Concurrency Problems

Problem	Also Called	What Happens	Example
Dirty Read	Write-Read (WR) conflict	T2 reads data written by T1 before T1 commits. If T1 aborts, T2 has invalid data.	T1 debits A but hasn't committed; T2 reads debited A value. T1 aborts → T2 has wrong balance.
Unrepeatable Read	Read-Write (RW) conflict	T1 reads data twice; T2 modifies it in between. T1 gets different values for same data.	T1 reads salary=50000. T2 updates to 60000. T1 reads again → gets 60000. Different result!

Lost Update	Write-Write (WW) conflict	Both T1 and T2 write the same data. One write overwrites the other.	T1 reads balance=1000, T2 reads balance=1000. T1 writes 1500, T2 writes 1200. T1's update lost!
Phantom Read	Phantom read anomaly	T1 runs query twice; T2 inserts/deletes rows in between. T1 sees different rows.	T1 counts students in CS=50. T2 inserts a new CS student. T1 counts again → gets 51.

8.6 Serializability

Conflict Serializability

Two operations conflict if they: (1) belong to different transactions, (2) access the same data item, and (3) at least one is a write.

Conflict Types:

- Read-Write (RW): Ti reads, Tj writes same item
- Write-Read (WR): Ti writes, Tj reads same item
- Write-Write (WW): Both Ti and Tj write same item

A schedule is conflict serializable if it can be converted to a serial schedule by swapping non-conflicting operations.

Precedence Graph (Serialization Graph)

Used to test conflict serializability:

- Create a node for each transaction.
- Draw edge Ti → Tj if an operation of Ti appears before a conflicting operation of Tj.
- If the graph has NO CYCLE → schedule is conflict serializable.
- If the graph HAS A CYCLE → NOT conflict serializable.

> Schedule: R1(A), R2(B), W1(A), R2(A), W2(B)
>
> Conflicts:
>
> T1 writes A, T2 reads A → T1 → T2 (WR conflict on A)
>
> Precedence Graph: T1 → T2 (no cycle)
>
> → Conflict Serializable! Equivalent to serial order: T1, T2

View Serializability

A more relaxed form. A schedule S is view serializable if it is view equivalent to some serial schedule. Two schedules are view equivalent if:

- Initial reads: If Ti reads initial value of Q in S, Ti must read initial value in serial schedule.
- Updated reads: If Ti reads Q after Tj writes Q in S, same must hold in serial schedule.

- Final writes: If Ti writes final value of Q in S, Ti must write final value in serial schedule.
- Every conflict serializable schedule is view serializable, but NOT vice versa.

8.7 Locking Protocols

Lock Types

- Shared Lock (S-lock / Read Lock): Multiple transactions can hold S-lock on same data simultaneously. Used for reading.
- Exclusive Lock (X-lock / Write Lock): Only one transaction can hold X-lock. No other transaction can read or write. Used for writing.

Lock Request	Shared (S) held	Exclusive (X) held
Shared (S) request	GRANTED ✓	DENIED ✗
Exclusive (X) request	DENIED ✗	DENIED ✗

Two-Phase Locking (2PL) Protocol

Each transaction must acquire all locks before releasing any. Two phases:

- Growing Phase: Transaction acquires locks. No lock is released.
- Shrinking Phase: Transaction releases locks. No new lock is acquired.

```
Growing Phase        Lock Point        Shrinking Phase

──────────────────────────────────────────────────────────────────────

Acquire S(A) ──────────────────────────── Release S(A)

Acquire X(B)                        Release X(B)

Acquire S(C)                      Release S(C)

    │            ▲
    └── Max locks held (Lock Point)


2PL guarantees conflict serializability!
```

- Basic 2PL: Guarantees conflict serializability but allows cascading rollbacks.
- Strict 2PL: All exclusive locks held until commit/abort. Prevents dirty reads.
- Rigorous 2PL: All locks (S and X) held until commit/abort. Simplest to implement.

Deadlock

Occurs when two or more transactions wait for each other to release locks, forming a circular wait.

```
T1 holds X(A), waiting for X(B)

T2 holds X(B), waiting for X(A)
```

→ Neither can proceed → DEADLOCK

Deadlock Prevention Strategies:

- Wait-Die: Older transaction waits; newer transaction is aborted (dies).
- Wound-Wait: Older transaction preempts (wounds) newer; newer transaction waits.
- Timeout: Abort transaction if it waits longer than a threshold.
- Deadlock Detection: Use Wait-For Graph; detect cycles; abort one transaction.

8.8 Introduction to PL/SQL

PL/SQL (Procedural Language/SQL) is Oracle's procedural extension of SQL. It adds programming constructs to SQL.

```
DECLARE
    v_salary NUMBER := 50000;
    v_name   VARCHAR2(50);
BEGIN
    SELECT Name INTO v_name FROM Employee WHERE EmpID = 101;
    IF v_salary > 40000 THEN
        DBMS_OUTPUT.PUT_LINE('High Earner: ' || v_name);
    ELSE
        DBMS_OUTPUT.PUT_LINE('Standard: ' || v_name);
    END IF;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Employee not found');
END;
/
```

PL/SQL Blocks consist of: DECLARE (variables), BEGIN (logic), EXCEPTION (error handling), END.

- Stored Procedures: Named PL/SQL blocks stored in DB. Called with EXEC.
- Functions: Return a value. Used in SQL expressions.
- Triggers: Automatically execute in response to INSERT/UPDATE/DELETE events.
- Cursors: Named pointers to a result set. Used to process rows one at a time.

End of DBMS Complete Study Guide

Covers all 8 units: Architecture | Relational Model | ER Model | Normalization | Algebra | Joins | SQL | Transactions